

Conception et développement d'IHM

Cours S5: Introduction à React Native

Contenu

- Présentation de React Native
- Le principe des « Vues » en développement mobile
- Les composants natifs / Les composants RN essentiels
- Mise en place de l'environnement de développement
- Traitement de la saisie de texte / Utilisation du ScrollView / Utilisation des List Views
- Comment styler les vues ?
- Comment définir la hauteur et largeur des vues?
- Layout avec Flexbox
- Comment afficher les images ?
- La manipulation des touches
- Comment naviguer entre les écrans ?
- Comment communiquer avec des APIs REST ?

Présentation de React Native

React Native (RN) est un Framework open-source permettant de **créer des applications Android et iOS à l'aide de React et des modules natives**. Avec React Native, vous utilisez **JavaScript** pour **accéder aux API** de votre plateforme (Android et iOS) et pour décrire l'apparence et le comportement de votre interface utilisateur à l'aide de **composants React**.

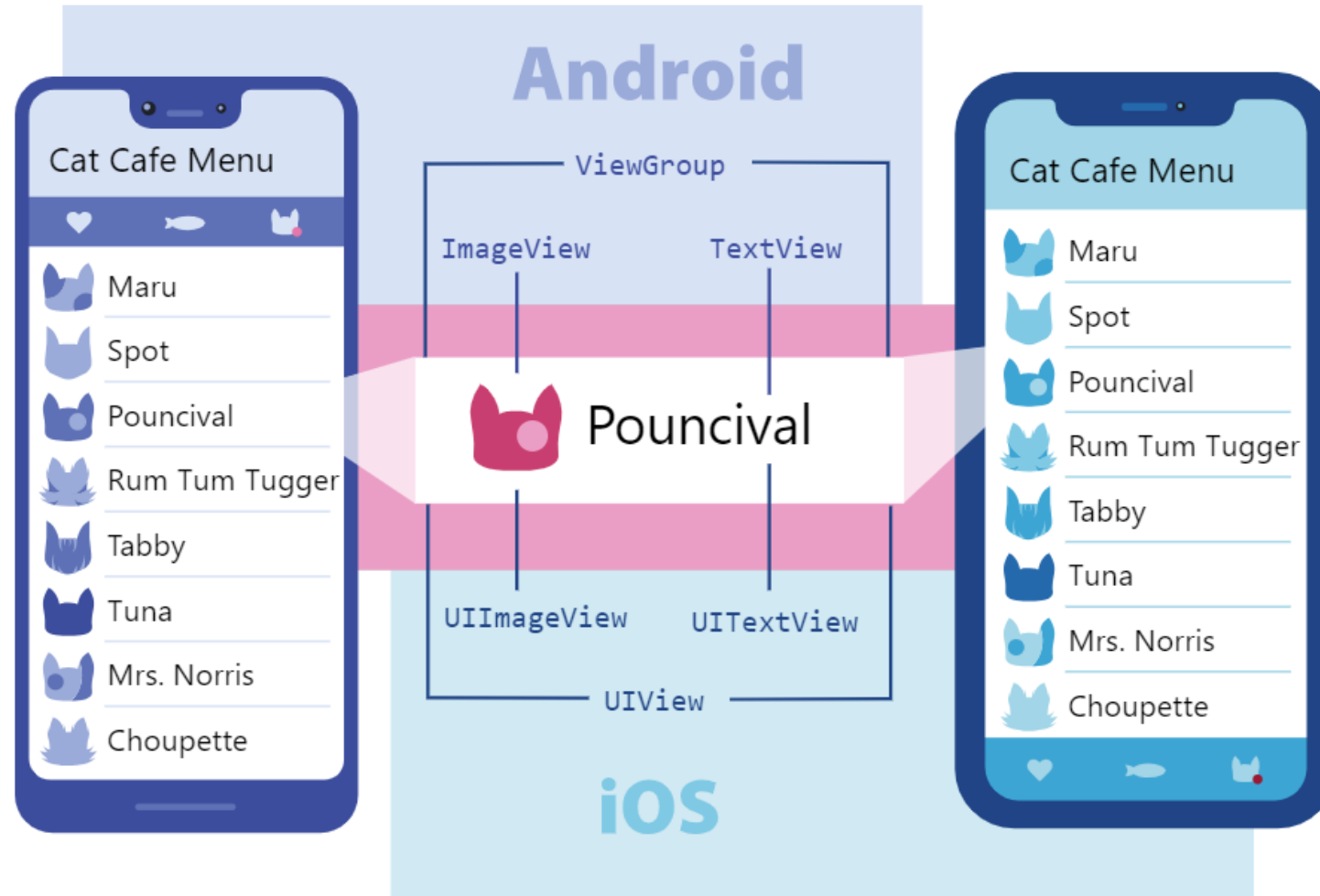
React Native a été publié pour la première fois par Facebook en tant que projet open-source en 2015. En quelques années seulement, il est devenu l'une des principales solutions utilisées pour le développement mobile. Il est utilisé pour créer certaines des principales applications mobiles au monde, notamment Instagram, Facebook et Skype.

Plusieurs raisons expliquent le succès mondial de React Native :

- Tout d'abord, en utilisant React Native, les entreprises peuvent créer du code **une seule fois** et l'utiliser pour créer à la fois leurs applications **iOS** et **Android**. Cela se traduit par d'énormes **économies de temps et de ressources**.
- Deuxièmement, React Native a été construit sur la base de **React** - une bibliothèque JavaScript, qui était déjà extrêmement populaire lorsque le framework mobile a été publié.
- Troisièmement, le framework a permis aux développeurs frontend, qui ne pouvaient auparavant travailler qu'avec des technologies basées sur le web, de créer des applications robustes et prêtes à la production pour les plateformes mobiles.

Le principe des « Vues » en développement mobile

Dans le développement Android et iOS, **une vue est l'élément de base de l'interface utilisateur** : un petit élément rectangulaire sur l'écran qui peut être utilisé pour afficher du texte, des images, ou répondre à l'entrée de l'utilisateur. Même les plus petits éléments visuels d'une application, comme une ligne de texte ou un bouton, sont des sortes de vues. Certains types de vues peuvent contenir d'autres vues.



Les composants natifs

Dans le développement Android, vous écrivez des vues en Kotlin ou en Java ; dans le développement iOS, vous utilisez Swift ou Objective-C. Avec React Native, vous pouvez **invoquer ces vues avec JavaScript en utilisant des composants React**. Au moment de l'exécution, React Native crée les vues Android et iOS correspondantes pour ces composants. Comme **les composants React Native sont soutenus par les mêmes vues qu'Android et iOS**, les applications React Native ont l'aspect, la convivialité et les performances des applications 100 % natives. Nous appelons ces composants soutenus par la plateforme des **composants natifs**.

React Native est livré avec un ensemble de composants natifs essentiels, prêts à l'emploi, que vous pouvez utiliser pour commencer à construire votre application. Il s'agit des composants de base de React Native.

React Native vous permet également **de créer vos propres composants pour Android et iOS** afin de répondre aux besoins spécifiques de votre application. RN dispose également d'un écosystème florissant de composants fournis par la communauté.

Les composants RN essentiels

React Native dispose de nombreux composants de base pour tout ce qui concerne les contrôles et les indicateurs d'activité. Ils sont tous documentés dans [l'API](#). Vous travaillerez principalement avec les composants de base suivants :

REACT NATIVE UI COMPONENT	ANDROID VIEW	IOS VIEW	WEB ANALOG	DESCRIPTION
<code><View></code>	<code><ViewGroup></code>	<code><UIView></code>	A non-scrolling <code><div></code>	A container that supports layout with flexbox, style, some touch handling, and accessibility controls
<code><Text></code>	<code><TextView></code>	<code><UITextView></code>	<code><p></code>	Displays, styles, and nests strings of text and even handles touch events
<code><Image></code>	<code><ImageView></code>	<code><UIImageView></code>	<code></code>	Displays different types of images
<code><ScrollView></code>	<code><ScrollView></code>	<code><UIScrollView></code>	<code><div></code>	A generic scrolling container that can contain multiple components and views
<code><TextInput></code>	<code><EditText></code>	<code><UITextField></code>	<code><input type="text"></code>	Allows the user to enter text

Exemple de code RN

```
import React from 'react';
import {View, Text, Image, ScrollView, TextInput} from 'react-native';

const App = () => {
  return (
    <ScrollView>
      <Text>Some text</Text>
      <View>
        <Text>Some more text</Text>
        <Image
          source={{
            uri: 'https://reactnative.dev/docs/assets/p_cat2.png',
          }}
          style={{width: 200, height: 200}}
        />
      </View>
      <TextInput
        style={{
          height: 40,
          borderColor: 'gray',
          borderWidth: 1,
        }}
        defaultValue="You can type in me"
      />
    </ScrollView>
  );
};

export default App;
```

Some text
Some more text



You can type in me

Mise en place de l'environnement de développement

La façon la plus simple pour mettre en place un environnement de développement RN est d'utiliser **Expo Go**. Expo est un ensemble d'outils et de services construits autour de React Native et, bien qu'il ait de nombreuses fonctionnalités, la plus pertinente pour nous est qu'il peut vous permettre d'écrire une application React Native en quelques minutes. Vous n'aurez besoin que d'une version récente de Node.js et d'un téléphone ou d'un émulateur.

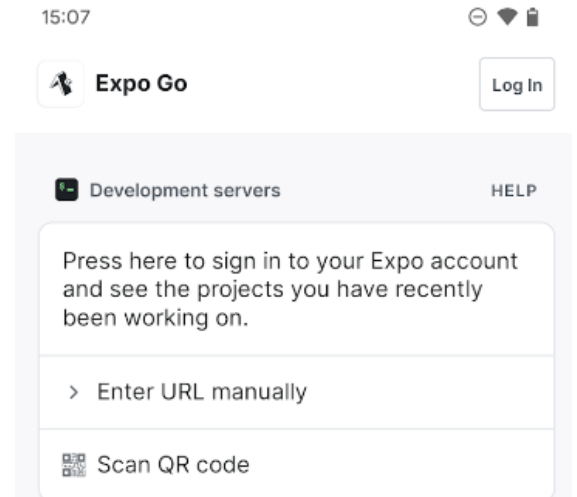
Exécutez la commande suivante pour créer un nouveau projet React Native appelé "hello-rn" :

```
npx create-expo-app hello-rn  
  
cd hello-rn  
npx expo start
```

Exécuter votre application React Native :

Installez l'application [Expo Go](#) sur votre téléphone iOS ou Android et connectez-vous **au même réseau sans fil** que votre ordinateur. Sur Android, utilisez l'application Expo Go pour scanner le code QR de votre terminal afin d'ouvrir votre projet. Sur iOS, utilisez le scanner de code QR intégré à l'application iOS Camera par défaut.

C'est possible aussi de tester les applications RN en local sur les émulateurs Android/iOS



Traitement de la saisie de texte

TextInput est un composant de base qui permet à l'utilisateur de saisir du texte. Il possède une propriété **onChangeText** qui prend une fonction à appeler chaque fois que le texte est modifié, et une propriété **onSubmitEditing** qui prend une fonction à appeler lorsque le texte est soumis.

Exemple :

```
import React, {useState} from 'react';
import {Text, TextInput, View} from 'react-native';

const PizzaTranslator = () => {
  const [text, setText] = useState('');
  return (
    <View style={{padding: 10}}>
      <TextInput style={{height: 40}} placeholder="Type here to translate!" onChangeText={newText =>
setText(newText)} defaultValue={text} />
      <Text style={{padding: 10, fontSize: 42}}>
        {text.split(' ').map(word => word && '🍕').join(' ')}
      </Text>
    </View>
  );
};

export default PizzaTranslator;
```

Utilisation du ScrollView

Le **ScrollView** est un conteneur de défilement générique qui peut contenir plusieurs composants et vues. Les éléments à faire défiler peuvent être hétérogènes, et il est possible de les faire défiler à la fois verticalement et horizontalement (en définissant la propriété horizontale)

Exemples :

```
import React from 'react';
import {Image, ScrollView, Text} from 'react-native';

const logo = { uri: 'https://reactnative.dev/img/tiny_logo.png', width: 64, height: 64 };

const App = () => (
  <ScrollView>
    <Text style={{fontSize: 96}}>Scroll me plz</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{fontSize: 96}}>If you like</Text>
    <Image source={logo} />
    <Image source={logo} />
  </ScrollView>
);
export default App;
```

Le ScrollView est idéal pour présenter un petit nombre d'éléments de taille limitée. Tous les éléments et toutes les vues d'un ScrollView sont affichés, même s'ils ne sont pas affichés à l'écran. Si vous avez une longue liste d'éléments qui ne peuvent pas tenir à l'écran, vous devriez plutôt utiliser une liste plate (FlatList).

Utilisation des List Views

RN fournit une suite de composants pour présenter des listes de données. En général, vous voudrez utiliser **FlatList** ou **SectionList**. Le composant FlatList affiche une liste défilante de données dynamique, mais structurées de manière similaire. FlatList convient bien aux longues listes de données, dont le nombre d'éléments peut changer au fil du temps. Contrairement au ScrollView, plus générique, FlatList n'affiche que les éléments actuellement affichés à l'écran, et pas tous les éléments à la fois.

Le composant FlatList nécessite deux propriétés : **data** et **renderItem**. data est la source d'informations pour la liste. renderItem prend un élément de la source et renvoie un composant formaté à afficher.

Exemple :

```
import React from 'react';
import {FlatList, StyleSheet, Text, View} from 'react-native';

const FlatListBasics = () => {
  return (
    <View>
      <FlatList
        data={[{key: 'Devin'}, {key: 'Dan'}, {key: 'Dominic'}]}
        renderItem={({item}) => <Text>{item.key}</Text>}
      />
    </View>
  );
};
export default FlatListBasics;
```

Comment styler les vues ?

Avec React Native, vous stylisez votre application à l'aide de **JavaScript** et **CSS**. Tous les composants de base acceptent une propriété nommée **style**. Les noms et les valeurs des styles correspondent généralement à la façon dont CSS fonctionne sur le web, à l'exception des noms qui sont écrits en utilisant la syntaxe **camelCase**, par exemple `backgroundColor` au lieu de `background-color`.

La propriété style peut être un simple objet JavaScript. Au fur et à mesure que la complexité d'un composant augmente, il est souvent plus simple d'utiliser **StyleSheet.create** pour définir plusieurs styles en un seul endroit. Voici un exemple :

```
import React from 'react';
import {StyleSheet, Text, View} from 'react-native';

const LotsOfStyles = () => {
  return (
    <View style={styles.container}>
      <Text style={styles.red}>just red</Text>
      <Text style={styles.bigBlue}>just bigBlue</Text>
    </View>
  );
};

const styles = StyleSheet.create({
  container: { marginTop: 50 },
  bigBlue: { color: 'blue', fontWeight: 'bold', fontSize: 30 },
  red: { color: 'red' },
});

export default LotsOfStyles;
```

Comment définir la hauteur et largeur des vues : dimensions fixes

La manière générale de définir les dimensions d'un composant est d'ajouter une largeur et une hauteur fixes au style. Toutes les dimensions dans React Native sont sans unité et représentent des pixels indépendants de la densité.

```
import React from 'react';
import {View} from 'react-native';

const FixedDimensionsBasics = () => {
  return (
    <View>
      <View style={{ width: 50, height: 50, backgroundColor: 'powderblue' }} />
      <View style={{ width: 100, height: 100, backgroundColor: 'skyblue' }} />
    </View>
  );
};

export default FixedDimensionsBasics;
```

Cette façon de définir les dimensions est courante pour le cas des composants où la taille doit toujours être fixée à un certain nombre de points et non calculée en fonction de la taille de l'écran.

Comment définir la hauteur et largeur des vues : Dimensions flexibles

Utilisez **flex** dans le style d'un composant pour que celui-ci s'étende et se rétrécisse dynamiquement en fonction de l'espace disponible. Normalement, vous utiliserez flex : 1, qui indique à un composant de remplir tout l'espace disponible, réparti de manière égale entre les autres composants ayant le même parent. Plus la valeur de flex est élevée, plus le composant occupera de l'espace par rapport à ses frères et sœurs.

```
import React from 'react';
import {View} from 'react-native';

const FlexDimensionsBasics = () => {
  return (
    <View style={{flex: 1}}>
      <View style={{flex: 1, backgroundColor: 'powderblue'}} />
      <View style={{flex: 2, backgroundColor: 'skyblue'}} />
      <View style={{flex: 3, backgroundColor: 'steelblue'}} />
    </View>
  );
};

export default FlexDimensionsBasics;
```

Comment définir la hauteur et largeur des vues : dimensions en pourcentage

Si vous souhaitez remplir une certaine partie de l'écran, mais que vous ne voulez pas utiliser la mise en page flexible, vous pouvez utiliser des valeurs de pourcentage dans le style du composant. À l'instar des dimensions flexibles, les dimensions en pourcentage requièrent un parent de taille définie

```
import React from 'react';
import {View} from 'react-native';

const PercentageDimensionsBasics = () => {
  return (
    <View style={{height: '100%'}}>
      <View style={{ height: '15%', backgroundColor: 'powderblue' }} />
      <View style={{ width: '66%', height: '35%', backgroundColor: 'skyblue' }} />
      <View style={{ width: '33%', height: '50%', backgroundColor: 'steelblue' }} />
    </View>
  );
};

export default PercentageDimensionsBasics;
```

Layout avec Flexbox

Un composant peut spécifier la disposition de ses enfants à l'aide de l'algorithme **Flexbox**. Ce dernier est conçu pour fournir une mise en page cohérente sur différentes tailles d'écran.

Vous utiliserez normalement une combinaison de **flexDirection**, **alignItems** et **justifyContent** pour obtenir la bonne mise en page.

Flexbox fonctionne de la même manière dans React Native que dans CSS sur le web, à quelques exceptions près. Les valeurs par défaut sont différentes : `flexDirection` est par défaut une colonne au lieu d'une ligne, `alignContent` est par défaut `flex-start` au lieu de `stretch`, `flexShrink` est par défaut 0 au lieu de 1, le paramètre `flex` ne prend en charge qu'un seul chiffre.

La [section flexbox](#) de la documentation officielle offre plusieurs exemples interactives des différentes propriétés flexbox à utiliser dans RN.

Comment afficher les images ?

React Native offre un moyen unifié pour gérer les images et autres ressources multimédias dans vos applications Android et iOS. Pour ajouter une image statique à votre application, placez-la quelque part dans l'arborescence de votre code source et référencez-la comme suit :

```
<Image source={require('./my-icon.png')} />
```

Le nom de l'image est résolu de la même manière que les modules JS. Dans l'exemple ci-dessus, le bundler cherchera my-icon.png dans le même dossier que le composant qui en a besoin.

Vous pouvez utiliser les suffixes @2x et @3x pour fournir des images pour différentes densités d'écran. Exemple :

```
.
├── button.js
└── img
    ├── check.png
    ├── check@2x.png
    └── check@3x.png
```

```
<Image source={require('./img/check.png')} />
```

Le bundler regroupera et servira l'image correspondant à la densité de l'écran de l'appareil. Par exemple, check@2x.png sera utilisé sur un iPhone 7, et check@3x.png sera utilisé sur un iPhone 7 Plus ou un Nexus 5. Si aucune image ne correspond à la densité de l'écran, la meilleure option la plus proche sera sélectionnée.

La syntaxe require décrite ci-dessus peut également être utilisée pour inclure statiquement des fichiers audio, vidéo ou des documents dans votre projet. Les types de fichiers les plus courants sont pris en charge, notamment les fichiers .mp3, .wav, .mp4, .mov, .html et .pdf

Images du réseau :

Contrairement aux ressources statiques, vous devrez spécifier manuellement les dimensions pour les images référencées sur le réseau.

```
<Image source={{uri: 'https://reactjs.org/logo-og.png'}} style={{width: 400, height: 400}} />
```

La manipulation des touches

Les utilisateurs interagissent avec les applications mobiles principalement par le toucher. Ils peuvent utiliser une combinaison de gestes, comme taper sur un bouton, faire défiler une liste ou zoomer sur une carte. React Native fournit des composants pour gérer toutes sortes de gestes courants, ainsi qu'un système de réponse gestuelle complet pour permettre une reconnaissance gestuelle plus avancée, mais le composant qui vous intéressera le plus probablement est le bouton de base.

Button fournit un composant de bouton de base qui est affiché de manière agréable sur toutes les plateformes. L'exemple minimal d'affichage d'un bouton ressemble à ceci :

```
<Button onPress={() => console.log('You tapped the button!'); } title="Press Me" />
```

Cela affichera une étiquette bleue sur iOS, et un rectangle arrondi bleu avec un texte clair sur Android. En appuyant sur le bouton, vous appellerez la fonction "onPress" qui, dans ce cas, affichera une fenêtre pop-up d'alerte. Si vous le souhaitez, vous pouvez spécifier une propriété "color" pour changer la couleur de votre bouton.

Si le bouton de base ne convient pas à votre application, vous pouvez créer votre propre bouton en utilisant l'un des composants "Touchable" fournis par React Native. Les composants "Touchable" permettent de capturer les gestes de tapotement et d'afficher un retour d'information lorsqu'un geste est reconnu. Ces composants ne fournissent pas de style par défaut, cependant, de sorte que vous devrez faire un peu de travail pour les rendre agréables.

Comment naviguer entre les écrans ?

Les applications mobiles sont rarement constituées d'un seul écran. La gestion de la transition entre plusieurs écrans est généralement assurée par ce que l'on appelle un navigateur. **React Navigation** fournit une solution de navigation directe, avec la possibilité de présenter des modèles courants de navigation par piles et par onglets sur Android et iOS.

Installation et configuration :

```
npm install @react-navigation/native @react-navigation/native-stack
npx expo install react-native-screens react-native-safe-area-context
```

Il faut maintenant envelopper l'ensemble de l'application dans NavigationContainer. Exemple :

```
// ./screens/HomeScreen.js
import { Text } from 'react-native';

export default function HomeScreen ({navigation}) {
  return (
    <Text>This is the home screen</Text>
  );
};
```

```
// ./screens/ProfileScreen.js
import { Text } from 'react-native';

export default function ProfileScreen ({navigation}) {
  return <Text>This is the profile screen</Text>;
};
```

Naviguer entre les écrans : navigation à pile

```
import * as React from 'react';
import {NavigationContainer} from '@react-navigation/native';
import {createNativeStackNavigator} from '@react-navigation/native-stack';
import HomeScreen from './screens/HomeScreen';
import ProfileScreen from './screens/ProfileScreen';

const Stack = createNativeStackNavigator();

const App = () => {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} options={{title: 'Welcome'}} />
        <Stack.Screen name="Profile" component={ProfileScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
};
```

Dans cet exemple, deux écrans (HomeScreen et ProfileScreen) sont définis à l'aide du composant Stack.Screen. Vous pouvez définir des options telles que le titre de chaque écran dans la partie options de Stack.Screen.

Chaque écran prend une propriété « component » qui est un composant React. Ces composants reçoivent une propriété appelée navigation, qui propose diverses méthodes pour établir des liens avec d'autres écrans. Par exemple, vous pouvez utiliser **navigation.navigate** pour aller à l'écran ProfileScreen.

Ce navigateur à pile native utilise les API natives : UINavigationController sur iOS et Fragment sur Android, de sorte que la navigation construite avec createNativeStackNavigator se comportera de la même manière et aura les mêmes caractéristiques et performance que les applications construites nativement au-dessus de ces API.

Naviguer entre les écrans : navigation par onglets

```
import * as React from 'react';
import {NavigationContainer} from '@react-navigation/native';
import {createBottomTabNavigator} from '@react-navigation/bottom-tabs';
import HomeScreen from '../screens/HomeScreen';
import ProfileScreen from '../screens/ProfileScreen';

const Tab = createBottomTabNavigator();

function TabsNavigation() {
  return (
    <NavigationContainer>
      <Tab.Navigator>
        <Tab.Screen name="Home" component={HomeScreen} options={{title: 'Welcome'}} />
        <Tab.Screen name="Profile" component={ProfileScreen} />
      </Tab.Navigator>
    </NavigationContainer>
  );
}

export default TabsNavigation;
```

Un autre style de navigation couramment utilisé consiste à faire apparaître des onglets sur la page et à naviguer de l'un à l'autre. Les onglets peuvent se trouver en haut ou en bas de l'écran.

Naviguer entre les écrans : navigation par menu

```
import 'react-native-gesture-handler';
import * as React from 'react';
import {NavigationContainer} from '@react-navigation/native';
import {createDrawerNavigator} from '@react-navigation/drawer';
import HomeScreen from './screens/HomeScreen';
import ProfileScreen from './screens/ProfileScreen';

const Drawer = createDrawerNavigator();

function DrawerNavigation() {
  return (
    <NavigationContainer>
      <Drawer.Navigator>
        <Drawer.Screen name="Home" component={HomeScreen} />
        <Drawer.Screen name="Profile" component={ProfileScreen} />
      </Drawer.Navigator>
    </NavigationContainer>
  );
}

export default DrawerNavigation;
```

Comment communiquer avec des APIs REST ?

La **consommation d'APIs REST** est très utilisé dans le développement mobile. Le plus souvent, les applications mobiles sont considérées comme des interfaces frontend, qui ont besoin de communiquer avec un backend. Comme dans React, ceci peut être fait avec **Fetch** ou avec des bibliothèques tierces comme **Axios**.

Exemple de consommation d'une API REST :

```
import React, {useEffect, useState} from 'react';
import {ActivityIndicator, FlatList, Text, View} from 'react-native';
const App = () => {
  const [isLoading, setLoading] = useState(true);
  const [data, setData] = useState([]);
  const getMovies = async () => {
    try {
      const response = await fetch('https://reactnative.dev/movies.json');
      const json = await response.json();
      setData(json.movies);
    } catch (error) { console.error(error); }
    finally { setLoading(false); }
  };
  useEffect(() => { getMovies(); }, []);
  return (
    <View style={{flex: 1, padding: 24}}>
      {isLoading ? <ActivityIndicator /> : <FlatList data={data} keyExtractor={({id}) => id} renderItem={({item})
=> <Text>{item.title}, {item.releaseYear}</Text> } /> }
    </View>
  );
};
export default App;
```